

SecureWave

Architecture, Algorithms and Assurance

Master's-level engineering analysis
Source version 1.0.0

Project author: Simon Parris
7 October 2026

A source-backed case study of a Linux WireGuard application: privilege separation, verified lifecycle, durable accounting and reproducible delivery.

Engineering documentation; not a peer-reviewed publication or security certification.

Contents

Abstract and research questions

1. Problem definition and requirements
2. System architecture
3. Security and identity model
4. Connection lifecycle algorithms
5. Durable usage accounting
6. Backend contracts and persistence
7. Correctness arguments and complexity
8. Evaluation methodology
9. Delivery workflow and reproducibility
10. Limitations, future work and conclusion
11. Experimental routing policy and validity

References and source traceability

The accompanying Markdown chapters and SVG figures are the source of this report. The reference manuscript supplies a structural example; its machine learning results are not SecureWave features.

Abstract and research questions

SecureWave is a Linux desktop VPN application with a Flutter interface, a FastAPI control plane, PostgreSQL persistence and a narrowly scoped privileged WireGuard helper. Its central engineering problem is keeping the user-facing connection state, the operating system's network state and the backend's usage ledger consistent despite partial failure. Authentication can succeed while provisioning fails; an interface can exist without a usable tunnel; traffic can continue after a GUI failure; and an accepted HTTP update can lose its acknowledgement. Treating these events as a single success flag would produce misleading connection and accounting claims.

This report derives the implementation's component model and algorithms from the source. It examines client-owned key generation, authenticated peer provisioning, evidence-based connection verification, process-owned teardown, durable local accounting and transactionally idempotent cumulative checkpoints. Formal notation describes the verification predicate and ledger invariants. Correctness arguments are conditional on stated trust, storage and database assumptions; they are not machine-checked proofs.

Evaluation combines backend contract tests, real PostgreSQL concurrency and migration tests, native helper tests, Flutter service and presentation tests, and a separate installed-product acceptance protocol. Automated test results establish selected contracts. They do not establish current production availability, reboot acceptance or cryptographic security. The report records these limits explicitly rather than inventing benchmark measurements.

Keywords: WireGuard; Flutter; Linux; privilege separation; state machines; idempotency; durable accounting; PostgreSQL; reproducibility.

Research questions

- RQ1: What observable evidence justifies displaying Connected?
- RQ2: How can a GUI control privileged networking without exposing a general privileged command interface?
- RQ3: How can retryable reports update a usage ledger without counting the same observed bytes twice?
- RQ4: What can be recovered after process failure or reboot, and which missing measurements must remain explicit?
- RQ5: Which evidence connects a source commit to an installed and tested product, and which claims require separate experiments?

The contribution is an integrated engineering design and a reviewable assurance argument. WireGuard provides the cryptographic transport [1]; this project does not introduce a new VPN protocol or cryptographic primitive.

1. Problem definition and requirements

1.1 Problem statement

A desktop VPN changes privileged operating-system state on behalf of an unprivileged user. The application must coordinate account identity, peer identity, interface ownership, routing, DNS and accounting. These states reside in different processes and persistence domains. No single database transaction can atomically include a GTK window, a kernel interface, a local journal and a remote HTTP response. The design therefore uses explicit transitions, verification, compensating cleanup and retryable accounting.

The supported implementation target is Ubuntu 24.04 ARM64 with a graphical session, systemd, WireGuard tools and the Linux Flutter runtime. The Flutter source has portable UI code; that does not establish working native VPN support on Android, iOS, Windows or macOS. Generated mobile scaffolding is not part of the supported product surface.

1.2 Functional requirements

ID	Requirement	Observable result
F1	Register and authenticate an account	Valid session reaches Home; incorrect password remains rejected
F2	Keep client private identity local	Configuration request transmits the public key only
F3	Provision an account-owned peer	Server parameters correspond to the requesting account and device
F4	Connect through the Linux helper	Fixed interface and expected peer appear in authoritative runtime output
F5	Verify usable VPN connectivity	Recent handshake, counters, route and changed public egress are observed
F6	Disconnect and recover	Interface/configuration are removed and network restoration is checked
F7	Record measured traffic durably	Local cumulative counters survive reporting interruption
F8	Avoid duplicate ledger effects	Repeated identical checkpoint is acknowledged without adding bytes again
F9	Expose uncertainty	Unavailable counters and measurement gaps are visible

1.3 Nonfunctional requirements

The helper should accept a finite protocol rather than arbitrary commands. Secrets should remain in protected stores and be excluded from source, screenshots, logs and public artifacts. UI rendering should consume authoritative application state rather than create a second VPN state machine. Backend updates should use bounded payloads and transactions. Releases should identify their source commit, tree state, architecture and helper contract. Documentation should allow a reviewer to trace an advertised property to code and a meaningful test.

These requirements do not imply a complete formal security audit. In particular, no claim is made that the application provides anonymity against a global observer, protects a compromised root account, prevents every possible IPv6 leak or reconstructs traffic never sampled before a crash.

1.4 Failure model and assumptions

The design considers expired authentication, provisioning failure, stale interfaces, missing handshakes, unreachable egress probes, counter resets, network interruptions, lost HTTP responses, concurrent checkpoint requests, GUI exit, daemon restart and reboot. A remote service can be unavailable for an unbounded time; successful reporting is therefore a conditional liveness property, not a fixed completion-time guarantee.

The trusted computing base includes the Linux kernel, root helper, installed network utilities, operating-system key store, backend and database. Local root can read or alter protected files and kernel counters. The usage ledger is operational metering under a trusted client/helper model, not a fraud-resistant billing system against a hostile device owner.

The storage argument assumes successful durable writes have the filesystem's documented persistence semantics. The accounting concurrency argument assumes PostgreSQL row locking and transactional commits. SQLite is useful for contract tests but cannot substitute for the production concurrency model [2].

1.5 Scope of the research analogy

The author's federated anomaly detection paper [9] provides a presentation example: define the problem, specify the model, present algorithms, explain evaluation and discuss limitations. The optional [routing experiment](#) adapts its multi-agent/XGBoost motivation to server selection, using a new simulator and explicit Bellman policy updates. It does not restore the anomaly classifier or implement federated training. The experiment is advisory: the established selector retains authority over every connection. An anomaly detection extension would require its own dataset, privacy model and evaluation.

2. System architecture

SecureWave component and trust model

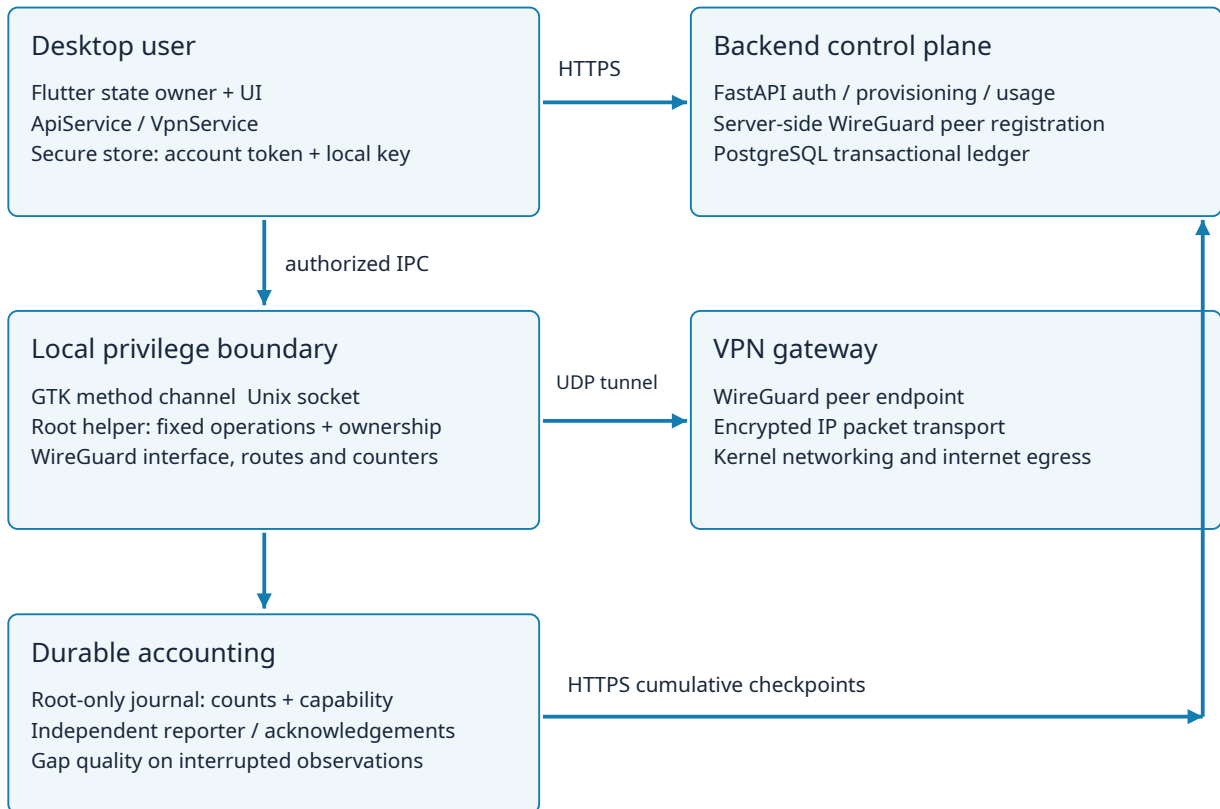


Figure 1. Control, network and accounting planes have distinct failure domains.

Figure 1. Components and trust boundaries

2.1 Three planes

The presentation and control plane begins in `securewave_app/lib/app.dart`. It owns authentication, connection transitions, recovery and usage polling. Widgets in `lib/ui/` receive state and callbacks. `ApiService` performs HTTP requests and maps endpoint-specific failures; `VpnService` orchestrates the native tunnel contract and validates runtime evidence.

The local network plane is a Flutter Linux method channel, a GTK runner, a Unix domain socket and the privileged helper daemon. The daemon executes approved WireGuard operations against a fixed interface. WireGuard and the operating system carry traffic; neither Flutter nor FastAPI encrypts each packet. The server registers peers and provides network parameters through the backend's WireGuard helper client.

The accounting plane spans the daemon's protected local journal, a separate reporter process and the backend usage service. GUI exit should not discard an already journaled checkpoint. Reporting continues independently using a session-scoped capability. The backend serializes accepted cumulative updates into session, event and peer records.

2.2 Components and responsibilities

Component	Responsibility	Explicit boundary
App state owner	Authentication and lifecycle transitions	Does not declare Connected from a successful config response
UI widgets	Render state and accessible actions	Do not provision peers or own tunnel state
ApiService	Auth, config and usage requests	Does not receive the client private key
VpnService	Native commands and verification	Requires observable tunnel evidence
GTK runner	Translate method calls to helper protocol	Does not accept arbitrary root shell commands
Helper daemon	Authorized network operations and durable samples	Fixed operations, caller identity and safe path checks
Usage reporter	Deliver pending journal entries and acknowledgements	Uses a per-session reporting capability
FastAPI routes	Validate requests, authenticate and authorize	Account-scoped resource access
Usage service	Serialize cumulative ledger updates	Database transaction and duplicate detection
PostgreSQL	Persist identities, peers, sessions and events	Production locking and crash consistency

2.3 Connection data flow

The signed-in client retrieves its account identifier and reuses or creates a WireGuard key pair in secure storage. It sends the public key to `POST /api/vpn/config`. The backend selects an eligible active server, checks ownership and address availability, and registers or reconciles the peer. The response supplies the server public key, endpoint, client address, allowed prefixes, DNS and keepalive information. It excludes a client private key.

The client combines these parameters with its local private key to form the temporary local configuration. It creates a backend usage session, then asks the helper to establish the interface with that session's accounting metadata. Verification interrogates the real runtime, route and egress. Only after this evidence passes and recording is confirmed does the state owner publish Connected.

Application HTTPS and WireGuard UDP traffic have distinct roles. HTTPS transports account and provisioning data. WireGuard transports tunneled IP packets. A working API response does not prove that the UDP endpoint is reachable or that routing changed. Conversely, an existing interface does not prove that the authenticated account's expected peer is active.

2.4 Trust boundaries and secret placement

The first boundary separates the GUI user from root networking. Socket permissions, group authorization and peer credentials constrain access. The second separates the device from the API, where bearer authentication and ownership checks protect provisioning and history. The third separates untrusted reporting requests from transactional ledger mutation.

The JWT remains an account credential in the client/API relationship. The reporting capability authorizes updates for one usage session and is stored in the root-protected journal; it is not the account JWT. The server stores its hash. Client private WireGuard keys remain local, while a temporary mode-0600 configuration necessarily exposes the private key to the trusted privileged networking component. This is a narrower boundary than claiming that no process ever accesses the key.

2.5 Source traceability

Review `app.dart`, `services/api_service.dart` and `services/vpn_service.dart` first. Then follow `linux/runner/my_application.cc`, `linux/helperd/securewave_helperd.cc` and `linux/helperd/usage_recorder.h`. Backend behavior is concentrated in `routes/auth.py`, `routes/vpn.py`, `routes/usage_recording.py`, `services/usage_metering_service.py` and the `models` directory. The [engineering walkthrough](#) links the corresponding files directly.

3. Security and identity model

3.1 Separate account, device, process and session identities

Let u be an account, k a client public key, p the owning GUI process and s a usage session. These identities serve different authorization decisions. The account authorizes provisioning and access to history. The device key identifies a WireGuard peer. The process owns a local tunnel instance. The usage capability authorizes reports for a particular session. Substituting one identity for another would broaden authority unnecessarily.

Client key storage is namespaced by account identifier. Reusing a key for the same account avoids creating a new peer on every Connect action. The backend rejects attempts to claim another account's peer identity. Revoked or inactive peers are not eligible for new active sessions. Operational account deletion, multi-device revocation and long-term rotation still require explicit product policies.

3.2 Authentication

Registration normalizes email addresses and validates the password contract. Passwords are stored as hashes through `hashing_service.py`. Login returns signed access and refresh tokens and sets cookies. The native application uses the access bearer. JWT claims include account subject, token type, expiry and the account's token generation. The decoder explicitly allows HS256 and checks the active account and current generation [3]. Logout increments that generation, invalidating prior access tokens for the account.

This is a signed token, not encrypted user metadata. Token confidentiality depends on protected storage and transport. Separate access and refresh signing secrets are required in production. Development can generate ephemeral secrets, which intentionally makes old sessions unusable after restart.

A login 401 means that the submitted credentials failed. A protected request's 401 can mean an expired or invalid session and routes the application back to sign-in. Registration is unauthenticated. These endpoint-specific semantics preserve useful error handling without suppressing genuine bad-password rejection. The current server registration flow does not verify mailbox ownership; its `email_verified` field must not be interpreted as independent email delivery evidence. Native automatic refresh is not asserted here.

3.3 Privilege separation

The GUI runs as the desktop user. The helper owns privileged WireGuard actions. Its request protocol has a fixed version and operation vocabulary. The helper checks socket peer credentials and authorized group membership rather than trusting a caller-supplied user identifier. Configuration validation rejects unsafe ownership, modes and filesystem paths. Approved executable invocations are constrained; the protocol does not expose an arbitrary shell endpoint.

The helper tracks the GUI process through a process descriptor where available. A `pidfd` refers to the process identity and avoids treating a reused numerical PID as the original owner [4]. A second GUI cannot silently adopt another window's active recorded tunnel.

Owner exit initiates measured finalization and teardown. These protections depend on the daemon being installed and running with the expected service and file permissions.

3.4 Configuration and transport

Local client key material uses Flutter secure storage backed by the Linux desktop's Secret Service integration [5]. The tunnel configuration exists only where the helper expects it, with restricted permissions, and is removed on safe teardown. Secrets are excluded from public evidence. Runtime checks use public peer keys, timestamps and counter values rather than printing a full WireGuard configuration.

HTTPS protects the control-plane exchanges when certificate validation and the deployment are configured correctly. TLS 1.3 is a relevant protocol reference [6], but this repository analysis does not claim that every deployed endpoint currently negotiates that version. WireGuard's authenticated encrypted transport and cryptokey routing are supplied by the protocol [1]. No custom cipher is added by SecureWave.

3.5 Threats and residual risks

Threat	Implemented response	Residual limitation
Account token replay after logout	Token generation check	A stolen currently valid token remains dangerous until invalidated
Cross-account peer claim	Ownership validation	Backend/root compromise bypasses application checks
Arbitrary privileged command	Fixed helper protocol and validation	Approved utilities and parser remain security-critical
GUI failure during tunnel	Process ownership, finalization and teardown	Abrupt system loss may precede final observation
Duplicate reporting	Capability check, digest and sequence transaction	Hostile trusted client can fabricate its own measured data
Secret disclosure in evidence	Redaction and protected local artifacts	Manual screenshots or external logs need careful handling

A complete audit would examine parser fuzzing, command timeout behavior, permission changes, dependency supply chain and network leak scenarios. The presence of controls is evidence of design intent and implementation, not a certificate that all attack paths have been eliminated.

4. Connection lifecycle algorithms

Connection lifecycle and evidence

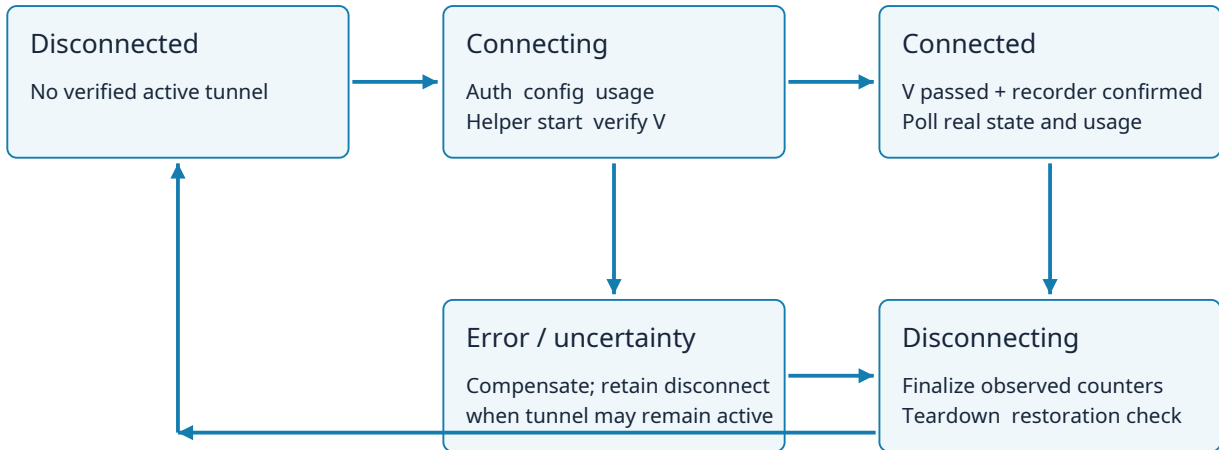


Figure 2. Success requires verification; cleanup failure retains possible-active state.

Figure 2. Verified connection lifecycle

4.1 State model

Let $Q = \{\text{disconnected}, \text{connecting}, \text{connected}, \text{disconnecting}, \text{error}\}$. The UI renders q in Q , while the helper reports the observed runtime. A separate flag records whether the tunnel may still be active after a failed operation. This prevents an error message from implying that privileged resources were removed. Transition controls are disabled while a lifecycle operation is busy. The state owner in `app.dart` remains authoritative; presentation widgets only map its values to text, controls and visual states.

The connection predicate can be expressed as:

```

V = I ∧ P ∧ H ∧ C ∧ R ∧ E
H = (handshake > 0) ∧ (-30 ≤ now - handshake ≤ 180 seconds)
C = counters_available ∧ (rx > 0) ∧ (tx > 0)
R = route_get(1.1.1.1).device = sw-wg
E = public_ipv4_after ≠ public_ipv4_before
  
```

I denotes an active expected interface; P denotes the configured server public key in its peer set. Both the runtime snapshot and a subsequent traffic-stat query must expose nonzero counters in both directions. Acceptance independently records both directions before and after generated traffic. The IPv4 route/egress checks are evidence of that path, not a proof that all applications, IPv6 traffic or alternate routing tables use the tunnel.

4.2 Algorithm 1: connect with compensation

```

Input: authenticated account u, current runtime, local secure store
Output: verified Connected, or Error with cleanup outcome
1 reject if another recorded tunnel already owns the interface
2 q ← connecting; unavailable counters; remember public IPv4 baseline
3 obtain account-scoped local WireGuard key pair (sk, pk)
4 params ← authenticated POST config(pk)
5 require installed helper availability and expected contract
6 s ← create usage session for the provisioned peer
7 mark tunnel as potentially active before requesting privileged creation
8 helper.connect(config(sk, params), session=s, expected_peer=params.pk)
9 require V from authoritative runtime and independent network observations
10 confirm usage recording; q ← connected; start runtime/usage polling
11 on failure: request teardown and verify disconnection
12 preserve Error and potential-active flag if cleanup cannot be verified
13 if the session never became connected: attempt failed-start finalization

```

The ordering places the accounting session before tunnel creation so that a successful privileged start has a reporting identity. Compensation covers failure after side effects begin. A failure to finish reporting does not erase the requirement to remove the tunnel. The API session and local interface are not committed atomically; orphaned failed-start sessions are a possible recovery case if both finalization and retry mechanisms are unavailable.

4.3 Algorithm 2: disconnection and network restoration

```

Input: potentially active owned tunnel, saved baseline
1 q ← disconnecting; stop GUI usage polling
2 helper finalizes measured usage and tears down the fixed interface
3 verify that the interface is absent and normal internet access is restored
4 clear saved baseline only according to the cleanup path's result
5 if verification succeeds: q ← disconnected; potential_active ← false
6 otherwise: q ← error; potential_active ← true; retain a disconnect action

```

The application's verification and the acceptance harness have different strengths. The application checks its supported restoration contract. The harness records before/after route tables, rules, resolver state and egress, then compares them. A harness comparison should normalize volatile fields if unrelated interfaces change during the experiment. Tailscale or other active network managers can otherwise create an attribution problem.

4.4 Recovery on launch

Session restoration first validates the stored access token. An expired session returns to sign-in. Runtime restoration is independent: if an interface exists, the app requires its saved baseline, local identity, expected peer and ownership/verification evidence. Missing or inconsistent evidence triggers safe cleanup rather than displaying Connected from cached UI state. Another window's tunnel cannot be silently adopted by the new process.

The daemon handles process exit independently of Flutter disposal callbacks. Callbacks are useful for orderly closure, but SIGKILL bypasses them. The process descriptor and journal support observed finalization when the owner dies. Reboot differs: the previous process and interface cease to exist, and journal recovery marks interrupted observations. A user's statement that the VM rebooted does not replace boot time, daemon startup, route and lifecycle evidence.

4.5 Observation limits

The predicate is sampled. An observed valid tunnel can fail immediately after verification; periodic polling updates the UI but cannot make it omniscient. The external egress probe introduces another dependency and can reject a usable tunnel when that service is unavailable. Handshake freshness bounds provide practical evidence rather than continuous cryptographic attestation. These tradeoffs favor truthful observable state over an optimistic success message after an API call.

5. Durable usage accounting

5.1 Measurement model

WireGuard exposes peer transfer counters. The helper samples the expected peer, accumulates changes and journals cumulative session totals. Let c_t be one raw counter, a_t its accumulated observed total and g a sticky gap flag. The rule is applied independently to sent and received directions:

```

 $\delta_t = c_t - c_{(t-1)}$       if  $c_t \geq c_{(t-1)}$ 
 $\delta_t = c_t$               if  $c_t < c_{(t-1)}$ , and  $g \leftarrow \text{true}$ 
 $a_t = a_{(t-1)} + \delta_t$ 
 $0 \leq a_t \leq 2^{63} - 1$ 

```

A reset exposes the bytes visible after reset but cannot reveal bytes between the last sample and that reset. Marking a gap is therefore more defensible than claiming complete measurement. Raw counters are bounded and parsed before use; arithmetic overflow is rejected. Counter totals measure WireGuard peer transfer, which can include protocol effects. They are not identical to a browser's downloaded file length or billable application payload.

5.2 Protected local journal

Each usage session has a restricted journal under `/var/lib/securewave/usage`. The directory is mode 0700, record files mode 0600, and record names and contents are validated. Records include session identity, capability, expected peer, boot identity, owner identity, cumulative counts, sequence and finalization state. The account JWT and client private key are not stored in this usage journal.

Writes use GLib's consistent and durable file replacement flags [7]. Under the documented filesystem assumptions, a successful write avoids exposing a partially overwritten record and requests persistence. This mechanism does not make failed writes succeed or eliminate hardware/filesystem faults. The daemon must still remove privileged networking when journal persistence fails; safety of teardown takes priority over claiming complete accounting.

5.3 Algorithm 3: sample and finalize

```

Input: active UsageRecord, expected peer, kernel runtime
1 read validated transfer counters for the expected peer
2 compute nonnegative direction deltas; mark gap on reset/unavailable data
3 update bounded cumulative totals and checkpoint sequence
4 durably replace the protected record
5 on disconnect/owner exit: quiesce and sample final observable counters
6 mark final reason and stop time; durably save final checkpoint
7 tear down interface and safely remove temporary configuration
8 release process ownership regardless of journal-write success

```

Actual finalization paths distinguish connect failure, owner exit and recovered interruption. Recovery of an unfinished record from a previous boot resets the raw-counter baseline and marks a gap; it does not invent a final kernel sample. Once replayed, a final record carries the best retained observations with an explicit quality indication.

5.4 Delivery protocol

The reporter periodically requests pending records from the helper and submits HTTPS checkpoints. The interval is two seconds in the current loop; it is an implementation parameter, not a reporting latency guarantee. The Authorization: UsageSession ... capability scopes reporting to one session. The backend compares its SHA-256 hash in constant time. The reporter acknowledges the helper after a successful accepted response. A response lost after commit leaves the checkpoint pending, so the next attempt repeats it.

The journal supplies retry persistence and the backend supplies idempotent effects. Neither component guarantees eventual delivery if the server remains unavailable forever, the disk is lost or the reporter is permanently disabled. Journal retention and a practical operational storage budget require explicit monitoring; this report does not assert a proven global bound for arbitrary offline duration and session churn.

5.5 Algorithm 4: transactional checkpoint acceptance

Let n be the submitted sequence, (S,R) cumulative totals and h the digest of the canonical complete payload. Let (n,S,R^*) describe the last accepted session state.

```

Input: session id, capability,  $n$ ,  $S$ ,  $R$ , final, quality and stop metadata
1 lock the session row inside a database transaction
2 verify capability hash, session existence and payload constraints
3 compute digest  $h$  over counts, sequence, flags, reason and stop metadata
4 if  $n \leq n^*$ :
5     accept as duplicate only when the stored event has the same digest
6     otherwise reject the conflicting replay
7 reject new mutations of an already finalized session
8 require  $S \geq S^*$  and  $R \geq R^*$ 
9  $\Delta S \leftarrow S - S^*$ ;  $\Delta R \leftarrow R - R^*$ 
10 atomically update session totals, peer totals and unique event record
11 persist sticky gap/verification quality and final state if requested
12 commit and return acknowledgement

```

A larger sequence may skip an earlier checkpoint: cumulative counts preserve the observed byte delta. A repeated exact earlier event can be acknowledged without effect. A changed payload using an old sequence is rejected. The digest includes quality and finalization metadata so that identical byte counts cannot conceal a conflicting semantic replay.

5.6 Exactly-once effects, conditional measurement completeness

The useful guarantee is that an accepted logical checkpoint has at most one ledger effect under the transaction and uniqueness assumptions. Delivery can occur multiple times. This is not an exactly-once network transport guarantee. The ledger may still be incomplete if a crash precedes sampling or persistence. Quality metadata distinguishes accepted accounting from complete observation.

Client measurements and server-side snapshots occupy separate fields. Adding both as if they were independent traffic would double-count the same flow. Server snapshots can support future reconciliation, but the current ledger protocol does not constitute a fully independent reconciliation or billing audit system.

5.7 Calendar-month account projection

The account view is a projection of accepted ledger events, not a second measurement system. For each cumulative session, the highest accepted sent and received values within a UTC calendar month are reduced by their highest preceding values. Summing these differences across account-owned sessions, plus legacy increment events, produces the monthly total. Device revocation does not remove historical events. Server snapshots remain excluded.

The Free allowance is 5,000,000,000 bytes, with decimal display units. The app adds only positive differences between its local observations and acknowledged session counters to the saved projection. Pending display observations are account-keyed; they cannot update the ledger. This preserves monotonic session presentation across disconnect and reauthentication without double-counting a checkpoint when the reporter catches up.

Accepted-event timestamps determine month attribution. This permits efficient aggregation with existing indexes but assigns late offline reports to their receipt month. Trusted interval-level timestamps would be needed for exact offline allocation across month boundaries. Client-side cap enforcement and backend provisioning checks also permit sampling/reporting overrun; they are not equivalent to independent server-side shaping. Detailed implementation and tests are in the [monthly usage guide](#).

6. Backend contracts and persistence

6.1 HTTP surface

Operation	Contract and authority
POST /api/auth/register	Unauthenticated validated registration; creates account and tokens
POST /api/auth/login	Credentials; returns access/refresh data; bad password is 401
GET /api/auth/me	Valid access identity; returns current account
POST /api/auth/logout	Valid access identity; advances token generation
POST /api/vpn/config	Bearer account; client public key; provisions server parameters
POST /api/vpn/usage/sessions/start	Bearer account; starts an owned peer's usage session
POST /api/vpn/usage/sessions/{id}/checkpoint	Session capability; cumulative replay-safe checkpoint
GET /api/vpn/usage/sessions	Bearer account; paginated history scoped to owner

Request and response schemas in the routes are the normative contract. The table summarizes the active design; reviewers should verify exact method and path names against `routes/usage_recording.py` and `routes/vpn.py` when changing clients. Configuration provisioning is idempotent with respect to an owned public key, but includes server-side reconciliation and can fail when live state is unavailable. Mock peer registration in unit fixtures is never production acceptance evidence.

6.2 Entity model

User owns peer identities and carries the authentication token generation. `VPNServer` describes endpoint, public identity, location and operational capacity. `WireGuardPeer` relates an account and server to a client public key and allocated address. `VPNConnection` represents a usage session with metering version, cumulative counters, sequence and recording quality. `VPNUsageEvent` records accepted checkpoint identity, cumulative counts and payload digest. Peer totals receive the delta calculated from those counts.

The relationship can be written:

```
User 1 → N WireGuardPeer
VPNServer 1 → N WireGuardPeer
WireGuardPeer 1 → N VPNConnection
VPNConnection 1 → N VPNUsageEvent
```

These cardinalities express the intended model, not a claim that every historic row has complete modern metadata. Legacy rows retain metering version 1 and legacy quality. Version 2 sessions finalize on recorded teardown, allowing a previous pending final report to

coexist with a new connection. Closing an old version 2 session merely because another starts would fabricate measurement endpoints and could discard its durable final checkpoint.

6.3 Transaction boundaries and concurrency

Start and checkpoint operations enforce ownership and serialize relevant rows. A checkpoint locks its connection with `SELECT ... FOR UPDATE` through SQLAlchemy. The event, session and peer delta updates commit together. In PostgreSQL, competing updates to that row wait until the owning transaction releases its lock [2]. The second identical request then observes the accepted sequence/digest and returns idempotently.

Two separate session rows can share a peer. Correct aggregate updates must avoid replacing peer totals with stale values; the implementation uses database-side increments where appropriate. Transaction ordering, event uniqueness and rollback behavior are critical review points. Tests deliberately submit two simultaneous identical checkpoints with totals exceeding 32-bit range. This evaluates the race that ordinary sequential SQLite tests cannot establish.

6.4 Additive migration

`scripts/migrate_usage_recording.py` provides the PostgreSQL usage migration. It adds columns conditionally, widens counters, preserves old rows, applies legacy defaults and adjusts the legacy active-device constraint. A transaction and advisory migration lock serialize migration attempts, with a bounded lock wait. Running it twice should not duplicate schema changes or alter preserved legacy identities.

The migration tests deliberately construct a legacy-shaped schema in a disposable database and apply the migration twice. They must never target a production database: they drop selected test columns to recreate that legacy shape. The migration's additive design is useful evidence but does not replace a deployment backup, permission review or staging rehearsal on the actual schema lineage.

6.5 Operational configuration

Production JWT and encryption secrets are mandatory configuration. Development ephemeral defaults are not stable production secrets. Request logging attaches identifiers and redacts common credential patterns; logging must not be used to dump full request bodies or WireGuard configurations. Health and readiness are different observations: a process can respond while its database or peer helper remains unusable.

History uses a bounded page size and an ID cursor. This keeps one response bounded while avoiding a full account history scan in client memory. Long-term event retention, rate budgets and service capacity require measurements and an operational policy; no throughput figure is inferred from passing tests.

7. Correctness arguments and complexity

7.1 Invariants

Symbol	Intended invariant	Enforcement point
A1	A peer's authenticated owner cannot be changed by another account's request	Provisioning ownership checks
L1	Connected follows successful current verification	App state owner and VpnService
L2	Cleanup failure retains possible-active state	Disconnect and compensation paths
M1	Accepted cumulative totals never decrease	Checkpoint validation under row lock
M2	Each accepted sequence has one canonical payload	Event identity and digest comparison
M3	Peer delta and session/event update commit together	Checkpoint database transaction
M4	Finalized state rejects new mutation	Final sequence guard
M5	A known measurement gap never becomes complete by a later report	Sticky quality update
S1	Local privileged action is restricted to authorized fixed operations	Helper protocol, peer credentials and path validation

These are source-derived design properties with corresponding tests. They are not unconditional guarantees under arbitrary kernel, root or database compromise. L1 concerns the transition instant; later network failure is detected by polling rather than prevented by the assertion.

7.2 Proposition 1: retry cannot add the same checkpoint twice

Assume PostgreSQL serializes competing updates to one session row; all relevant ledger effects occur in the same transaction; and the event's logical identity and digest remain intact. Consider two identical submissions (n, S, R, h) . The first successful transaction changes the accepted state from (n, S, R) to (n, S, R) and adds $(S-S, R-R^*)$ to the peer. The second transaction reads n as the current sequence, matches the retained digest and returns idempotently before performing an increment. If the first transaction rolls back, its ledger effects are absent and a later submission can perform them once.

An HTTP acknowledgement can be lost after commit without changing this argument. The retry observes committed state. The argument would fail if the peer update occurred outside the transaction or if a stale read could bypass serialization. This is why the real PostgreSQL race test matters.

7.3 Proposition 2: cumulative updates tolerate omitted intermediate reports

For accepted checkpoints indexed 0 through m , define $\Delta S_i = S_i - S_{(i-1)}$, with $S_0 = 0$. The total added is:

$$\sum_{(i=1..m)} \Delta S_i = \sum_{(i=1..m)} (S_i - S_{(i-1)}) = S_m$$

The same telescoping identity holds for received bytes. Skipping an intermediate sequence does not change the final observed total, provided the later cumulative value contains its observations and is accepted. It does not repair unobserved traffic. A decreasing total cannot be interpreted as a new positive delta and is rejected.

7.4 Proposition 3: quality remains conservative

Let g_i indicate that the client reported a gap or the stored session already had gap quality. The service chooses gap whenever g_i is true. Induction on accepted updates shows that once gap is stored, no later accepted update can make it complete. Finalization freezes the accepted endpoint. This is a conservative record of known uncertainty, not proof that an unflagged session contains every packet: undetected loss remains a threat to validity.

7.5 Lifecycle safety and liveness

The lifecycle algorithm prevents a config response alone from publishing Connected. Compensation requests teardown after side effects may have begun. The helper finalizes before removal when observable counters are available, but removal remains necessary if persistence fails. These choices prioritize network-state safety and truthful quality reporting.

Conditional liveness requires that the helper can execute teardown, the network eventually reaches the reporting server, pending records remain durable, and the reporter continues running. Permanent network partition or storage loss violates those assumptions. No algorithm can guarantee successful remote acknowledgement when the receiver is permanently unreachable.

7.6 Complexity model

Let P be the number of peers returned by a local runtime query, J the retained local journal records, H an account's history rows and B the bounded page size. Counter parsing and peer selection are $O(P)$; the desktop's expected single-peer interface makes this practically small without changing the general bound. Returning pending journals requires $O(J)$ inspection in the current helper and caps each returned batch at 16. Sampling one session uses $O(1)$ arithmetic and bounded record storage. Rewriting a journal costs filesystem I/O, which is not captured by an arithmetic-only complexity statement.

A checkpoint performs a bounded number of row queries and updates. Index lookup costs depend on table size and the database plan; lock waiting depends on contention and cannot be bounded by saying $O(1)$. History returns $O(B)$ records after a filtered ordered query; suitable indexes determine whether lookup remains efficient as H grows. Event retention

grows with accepted checkpoints and needs an operational retention decision.

The report does not claim measured latency, throughput, memory savings or energy efficiency. Such claims would require controlled benchmarks and an explicit hardware/software environment.

8. Evaluation methodology

8.1 Evidence hierarchy

Source analysis identifies mechanisms. Unit and contract tests examine chosen inputs under controlled fixtures. Native tests exercise helper parsing, ownership, accounting and cleanup with controlled process/network substitutes. Real PostgreSQL tests examine row-lock races and migration preservation. GUI goldens establish presentation expectations. Installed acceptance observes the actual kernel, helper, network and backend. Each layer answers a different question; passing one does not imply passing all others.

Layer	Main question	Limitation
Python contract suite	Are validation, ownership and replay semantics correct for tested cases?	Fixtures simulate peer infrastructure
PostgreSQL suite	Does concurrent replay count once; is legacy migration additive?	Disposable database, not production load
Native helper suite	Do authorization, parser, journal and cleanup cases behave correctly?	Controlled commands, not live tunnel proof
Flutter service suite	Are lifecycle and API errors interpreted correctly?	Mock channels and HTTP adapters
Flutter UI/goldens	Are controls, layout, text scaling and state displays consistent?	Rendered fixture state, not network evidence
Package provenance	Do installed bytes identify the intended clean source and architecture?	Identity and integrity only
Live/reboot acceptance	Does the installed product complete the real lifecycle?	Environment-specific observation

8.2 Reproducible software checks

The consolidated checks use Python 3.12, Flutter 3.41.4/Dart 3.11.1, pinned Python requirements and Flutter's lockfile. `make test-backend`, `make test-flutter`, `make test-native` and `make check` are the review entrypoints. The PostgreSQL tests require `SECUREWAVE_TEST_DATABASE_URL` pointing only to a disposable database; without it, those tests are explicitly skipped. CI runs them against its own service. Results and environment limitations belong in [current state](#), not a mutable assertion in each chapter.

Test values exceeding 32-bit integer range assess the widened accounting contract. Conflicting replay, decreasing totals, final mutation and cross-account requests are negative cases. Native tests examine ownership and finalization failures. Flutter tests include endpoint-aware 401 handling, malformed helper output, missing handshake, absent counters, route/egress failure, transitions, keyboard focus and layouts at larger text scales.

8.3 Installed acceptance protocol

The retained harness is `scripts/securewave_post_reboot_acceptance.py`. It interacts with the installed GUI through accessibility actions and records independent observations to a private existing evidence directory. A baseline captures route tables, rules, DNS, public egress, daemon state and absence of stale interface/configuration. After authenticated Connect, verify the expected peer, fresh handshake, interface, protected config mode, route, DNS and changed egress. Generate bounded test traffic and confirm counter growth. Disconnect and compare restoration; reconnect and repeat; finish disconnected.

The script's lifecycle command does not independently verify the final production usage ledger. Complete accounting acceptance additionally requires owner-scoped backend history or a separately authorized database observation, matching the session, sequence, counters, final reason and quality. Registration of a disposable live account is an explicit separate operation, used only when authorized; no credentials are printed or included in evidence.

8.4 Crash and reboot experiments

For GUI SIGKILL, establish a verified connection, generate traffic, record the session, terminate only the owning process, then observe helper finalization, interface removal, reporter acknowledgement and final history. For reporter outage, retain unacknowledged journals, restore delivery and test that replay adds no duplicate bytes. For reboot, record boot identity and a pre-reboot baseline; after startup, verify the new boot, enabled/active services, recovery quality, absence of stale networking, cold launch and a fresh authenticated connect/disconnect/reconnect sequence.

Historical usage-candidate evidence is archived with its original source and version. On 6 October 2026 the installed candidate's post-reboot baseline and cold launch passed, but the saved session had expired. Authenticated post-reboot VPN acceptance remains pending sign-in. Those observations do not establish acceptance of the consolidated 1.0.0 source or package.

8.5 Threats to validity

Internal validity is affected by other network managers, cached session state, clock drift and external probe failure. Construct validity depends on using kernel transfer counters rather than interpreting a UI animation as traffic. External validity is limited to the tested Ubuntu ARM64 environment; x86_64, mobile, different DNS managers and hostile local users need separate evidence. Fixture tests may omit timing races and production permissions. A small concurrency experiment is evidence for one race, not a full scalability study.

No fabricated performance table is presented. A future benchmark should compare the normal link and tunnel under the same route, server, CPU load and traffic generator; report repeated-trial throughput and latency distributions, packet loss, handshake time, CPU use and confidence intervals. Recovery tests should vary outage duration and inject faults before/after each durable write and database commit. Such experiments would strengthen the assurance case.

9. Delivery workflow and reproducibility

9.1 Source organization

The repository has one active branch, `master`. Integrated feature histories remain reachable through merge commits and archive tags. Recruiters can begin with `docs/portfolio/`; engineers can follow `docs/development/`; the formal analysis lives here. `docs/archive/` preserves earlier release and handoff records without presenting their version numbers as current source metadata. Private credentials, local build outputs and generated unsupported platform scaffolding are excluded from the public source surface.

The 1.0.0 application version is distinct from helper contract 15, helper request protocol 1 and accounting protocol version 2. Renaming the application does not reset those compatibility identifiers. Original historical packages continue to identify their actual 4.0.0+... versions. The existing historical `v1.0.0` Git tag is not silently moved; a new candidate can use a distinct tag.

9.2 Source-to-package chain

A defensible release chain is:

```

reviewed master commit
→ clean detached checkout
→ locked dependency resolution and release build
→ package metadata + embedded source/tree/architecture/contract markers
→ checksum and source sidecars
→ explicitly identified candidate asset
→ installation on the supported device
→ cold launch + authenticated real lifecycle + reboot proof
→ authorized public publication and download verification

```

The Debian builder packages the Flutter bundle, helper, wrapper, reporter, systemd units, launcher and icon. The source marker identifies the exact commit; the tree-state marker distinguishes a clean build from uncommitted inputs. An ARM64 ELF check confirms architecture rather than relying solely on the filename. Checksums prove the identity of transferred bytes, not their correctness or origin independently of the embedded marker.

9.3 Version normalization

Application metadata is normalized to 1.0.0 in `VERSION`, the Flutter pubspec and backend release metadata. Debian regards 1.0.0 as lower than 4.0.0+12, so an installed older-labelled candidate may require an explicit version downgrade operation. This should be documented rather than disguised by retaining a higher hidden package number. Changing source metadata alone does not replace a previously installed package or existing public download.

9.4 CI and reviewer workflow

Continuous integration runs for master pushes and review requests, with read-only repository permissions and no production credentials. Jobs validate backend contracts against disposable PostgreSQL, Flutter static analysis and tests, native helper tests, source versions, documentation links and PDF rebuilding. A CI pass is recorded for its exact SHA. It does not install the desktop app on the user's VM or alter live peer infrastructure.

The release workflow is documented rather than automated into an uncontrolled production deployment. Database migration, server provisioning and public publication are separate operational actions requiring their own environment and evidence. The repository keeps their scripts inspectable while avoiding the suggestion that a recruiter needs production access to review the code.

9.5 Reproducibility limits

Pinned dependencies and provenance improve repeatability. They do not guarantee bit-identical binaries across all machines: toolchain versions, environment, timestamps and platform libraries can affect bytes. The report renderer embeds fonts, uses deterministic PDF metadata and stores SVG source figures; reviewers can rebuild the document without a remote service. Any deterministic-output claim should be verified by comparing hashes from two runs in the same pinned environment.

The public validation record distinguishes local software checks, package provenance, installed acceptance and serving evidence. Raw secret-bearing logs and private evidence directories are not copied into GitHub to manufacture a more impressive-looking evidence bundle.

10. Limitations, future work and conclusion

10.1 Current limits

The supported implementation is a Linux WireGuard application. Portable Flutter widgets and protocol catalog responses do not establish working OpenVPN, IKEv2 or cross-platform networking. The UI presents authoritative supported state rather than fabricated server lists or throughput. Historical production measurements belong to their original commit/package, while the consolidated 1.0.0 candidate needs its own installed acceptance.

The verifier observes IPv4 routing and public egress. Comprehensive IPv6, split-routing, captive portal, suspend/resume, resolver conflict and leak tests remain further work. A hardened kill switch is a separate claim requiring firewall-state and failure-injection evidence. Disconnect restoration does not by itself prove continuous leak prevention during every transition.

Usage is sampled and supplied by a trusted local helper. It can be incomplete across unsampled crashes, resets or disk faults. Root compromise can alter measurements. Durable replay prevents duplicate accepted effects but does not establish tamper-proof billing or indefinite offline storage capacity. A business policy for retention, deletion, reconciliation and privacy is needed before interpreting the ledger as a commercial billing record.

Authentication currently permits immediate registration/login without a mailbox verification gate. Native token refresh, self-service key rotation, device revocation UX and account lifecycle policies should be evaluated as separate features. They should not be assumed from the presence of refresh tokens or an `email_verified` response field.

10.2 Prioritized research and engineering work

Priority	Work	Evidence needed
1	Finish 1.0.0 installed/reboot acceptance	Fresh exact-package lifecycle, recovery and final ledger observations
2	Network failure and leak matrix	IPv4/IPv6, DNS, sleep/resume, unreachable endpoint, firewall inspection
3	Fault-injected journal/reporting recovery	Crash at write/commit boundaries; exact totals and quality flags
4	Parser and privilege-boundary audit	Fuzzing, permission mutation, path attacks and timeout tests
5	Retention and reconciled metering	Storage growth, server/client disagreement policy, privacy/deletion model
6	Capacity and performance study	Repeated trials, resource metrics and uncertainty intervals
7	Additional platforms	Native implementations and real device acceptance per platform

A future anomaly detection extension inspired by the author's research would need a new specification. Traffic metadata collection can itself affect user privacy. Before adding federated learning, define which observations leave the device, consent, adversarial participants, model-update protection and a reproducible baseline. No inference model or detection accuracy is claimed in the current application. A separate [MARL/XGBoost routing experiment](#) trains on simulator episodes and observes measured inputs in shadow mode. Its frozen synthetic holdout does not establish production routing gains, and the combined policy did not outperform XGBoost alone. One physical production host cannot validate alternative-server selection. Backend ICMP probes and normalized load averages also differ from the simulator's client latency/congestion variables; fresh timestamps do not remove that domain mismatch.

10.3 Conclusion

RQ1 is addressed by observable interface, peer, handshake, counter, route and egress checks before the Connected transition. RQ2 is addressed by an unprivileged GUI and a constrained credential-checked helper. RQ3 is addressed by durable cumulative reports and transactional sequence/digest idempotency. RQ4 is addressed by process ownership, persisted checkpoints and explicit gap quality where recovery cannot reconstruct observations. RQ5 is addressed by commit and package provenance followed by separate installed and public delivery evidence.

The engineering lesson is that reliable VPN behavior depends on coordinating several failure domains while keeping their evidence distinguishable. The source provides mechanisms and tests for that coordination. Remaining runtime, security and scalability

questions require the targeted experiments described above rather than stronger wording about existing results.

11. Experimental routing policy and validity

11.1 Objective and deployment contract

The optional extension studies whether predicted future utility and learned action values can improve server selection. It adapts the author's multi-agent XGBoost motivation [9] to a new task. It does not reuse anomaly labels or the paper's accuracy results. The original reward/trust and ensemble scripts do not contain the Bellman Q-learning implemented in this experiment.

Let C be the account's current eligible server set and $b(C)$ the established selector. The deployed contract is:

```
selected(C) = b(C), independently of any shadow recommendation h(C)
```

Thus failures or differences in h cannot change provisioning authority. No activation flag exists in this version. Explicit choices retain their normal eligibility checks; zero candidates return unavailable and one candidate skips the experiment. The inference worker is separate from the API request process.

11.2 Learning specification

Three simulated clients maintain independent Q tables while sharing four congested server resources. The feature vector is ordered latency, loss, jitter and normalized load. An XGBoost regressor estimates post-action utility from pre-action features generated under randomized offline actions.

The simulator utility weights are $0.45 \exp(-\text{latency}/180)$, $0.25(1-\text{loss})$, $0.15 \exp(-\text{jitter}/35)$ and $0.15(1-\text{load})$, bounded in $[0,1]$. Switching subtracts 0.03 from an agent's reward.

Independent Q updates use learning rate 0.18, discount 0.9 and zero continuation at terminal transitions. Three actions represent highest predicted utility, lowest load and the baseline. Quantized context captures quality/load relationships rather than identities.

Visit-weighted aggregation forms one advisory table from the independent learners. Unknown contexts use explicitly labelled XGBoost-only ranking. Training with four candidates does not validate other fleet sizes; those sizes have unseen context. The deployed single-host fleet therefore supplies no evidence of learned alternative-server selection.

For n candidates, T trees of depth d and k visited states, offline prediction costs approximately $O(nTd)$; deterministic ranking costs $O(n)$. The tabular update costs $O(1)$ for three actions, and policy storage is $O(k)$. API observation is bounded by candidate count and fixed file-size limits; it imports no native ML library. These bounds do not promise a wall-clock deadline for filesystem I/O, which remains an operator-controlled dependency in shadow mode.

11.3 Experimental controls and outcomes

Episode seeds and regime groups are disjoint across train, validation and test. All model fitting and policy updates occur on training episodes. Both holdouts use the frozen artifacts and paired random streams; neither is an early-stopping set. The manifest records the split, dependencies, feature order and hashes.

The seed-42, 80-training-episode run produced 5,760 XGBoost observations and 1,920 updates per agent. Over 20 frozen test episodes, mean utility was 0.525263 for the simulator latency baseline, 0.545234 for XGBoost alone and 0.533659 for the combined policy. The combined method trails the simpler ablation. No statistical significance or real-world routing benefit is inferred. Production also uses a different baseline ordering, so these values are not its uplift.

11.4 Construct validity and required future evidence

Real collection measures collector-to-server ICMP mean RTT, loss fraction and RTT standard deviation, together with load average per CPU. These have the same numeric shape as simulator features but different semantics. Five ICMP probes give coarse loss resolution; load average includes blocked/runnable work rather than simulator congestion. The legacy monitor's estimated jitter and simulated load are explicitly excluded from measured evidence.

An external probe on 7 October received no ICMP replies. No missing RTT was filled with a default and no measured snapshot was fabricated. Freshness, measurement method, perspective, normalized predictions and artifact hashes are checked before advisory evaluation. None establishes model calibration.

Activation would require independent physical nodes, measured future outcome targets, calibration, time-separated evaluation, baseline/ablation comparison, uncertainty estimates and bounded switching/failure experiments. A new review would also need to specify privacy, retention and authority changes. The current extension remains an implementation experiment with fail-open shadow behavior.

The [operator and reproducibility guide](#) provides commands, schemas and data provenance. App lifecycle evidence remains separate from this model's validity.

References and source traceability

External sources accessed 6 October 2026. The report's implementation claims come from this repository; external references explain the underlying protocols and platform semantics. Links remain useful without reproducing the referenced papers or standards.

- [1] J. A. Donenfeld, WireGuard: Next Generation Kernel Network Tunnel, protocol paper, revision 1 June 2020. [Original paper](#).
- [2] PostgreSQL Global Development Group, Explicit Locking, PostgreSQL documentation. [Row and transaction locks](#).
- [3] M. Jones, J. Bradley and N. Sakimura, JSON Web Token (JWT), RFC 7519, May 2015. [RFC 7519](#).
- [4] Linux man-pages project, pidfd_open(2). [Process descriptors](#).
- [5] freedesktop.org, Secret Service API. [Specification](#).
- [6] E. Rescorla, The Transport Layer Security (TLS) Protocol Version 1.3, RFC 8446, August 2018. [RFC 8446](#).
- [7] GLib project, FileSetContentsFlags and g_file_set_contents_full. [Flags](#) and [write semantics](#).
- [8] Flutter project, Writing custom platform-specific code. [Platform channels](#).
- [9] S. Parris et al., Decentralized Federated Detection of Network Anomalies in Mobile Networks, author's supplied IEEE-format manuscript, 2025. [Reference manuscript](#).
- [10] PyJWT project, Changelog, version 2.15.0. [Dependency release notes](#).

Code-to-claim map

Claim	Primary source	Verification endpoint
Endpoint-specific auth behavior	app API service; routes/auth.py; jwt_service.py	Flutter API tests; backend auth contracts
Client-owned peer identity	app.dart; routes/vpn.py	test_vpn_client_owned_config.py
Verified connection	vpn_service.dart	Flutter WireGuard service tests; installed harness
Privileged operation boundary	my_application.cc; securewave_helperd.cc; securewave-wg-quick	Native helper and wrapper tests
Durable measured counters	usage_recorder.h; securewave-usag e-reporter.py	Native recorder cases; crash acceptance
Idempotent accounting	usage_metering_service.py; usage_recording.py	test_usage_recording.py; test_usage_postgres.py
Additive legacy migration	migrate_usage_recording.py	Real PostgreSQL migration test
Version/provenance	VERSION; release_metadata.py; pubspec.yaml; build_deb.sh	Repository checks; extracted package markers

For runnable commands and exact current results, see the [testing guide](#) and [validation record](#). Commit history and archived records retain earlier design and release evidence.